

Supplementary Material

Anonymous Author(s)

1 Implementation Details

All experiments are conducted on an NVIDIA A100 80GB PCIe GPU, using 12 CPU cores with 7.5 GB memory each. The software environment includes CUDA 11.8, PyTorch 2.6.0, and Hugging Face Transformers 4.49.0. To ensure stability and reproducibility, we initialize random states using multiple seeds for Python, NumPy, PyTorch, and CUDA.

Due to the differing nature of encoder-only and decoder-only architectures, we adopt tailored fine-tuning strategies. Encoder-only models use a custom training loop with gradient accumulation for better training control. Decoder-only models (LLMs) are fine-tuned using HuggingFace’s SFTTrainer, with 4-bit quantization via BitsAndBytesConfig for memory efficiency, and LoRA-based parameter-efficient fine-tuning (PEFT) to reduce training overhead while maintaining performance. Fine-tuning results for encoder-only models are averaged over 5 random seeds [42, 123, 456, 789, 1024]; decoder-only models are averaged over 3 seeds [42, 123, 456] due to computational constraints. We also evaluate zero-shot learning by designing a baseline prompt that relies on the model’s pre-trained knowledge without additional training, and few-shot in-context learning by including three task-specific examples within the input prompt. The three examples are selected from the training set using class-balanced sampling (one verifiable, two non-verifiable) with a fixed random seed (42), ensuring deterministic and reproducible example selection across all experiments. The prompt templates are shown in Table 4.

2 Parameter Configurations

For encoder-only models, hyperparameters are optimized on the CT22 development set using the Optuna framework [1] with Bayesian optimization via the Tree-structured Parzen Estimator (TPE) sampler. Each configuration undergoes 20 independent trials, with a MedianPruner strategy to terminate underperforming trials early. Baseline and ContextClaim configurations are optimized independently to ensure a fair comparison. The resulting hyperparameters are applied to both CT22 and PoliClaim without further dataset-specific tuning, so the PoliClaim results reflect cross-dataset transfer of hyperparameter choices. The optimized configurations for the ContextClaim variants are reported in Table 1. For decoder-only models, fine-tuning configurations are selected through empirical tuning to balance computational efficiency and performance, as detailed in Table 2.

In addition to model-specific hyperparameters, the context retrieval component uses fixed parameters throughout all experiments: $p = 5$ candidate extracts per query, relevance score weights $\alpha = 0.8$ and $\beta = 0.2$, and a filtering threshold $\theta = 0.5$ for MISC (and DISEASE) entities. These values were fixed throughout all experiments based on the design rationale and none of these values were tuned on any development set.

Table 1: Hyperparameter configurations for encoder-only ContextClaim models.

Model	Param.	CC-G4o	CC-M
BERT	Epochs	8	8
	Batch	20	20
	Dropout	0.19	0.19
	Warmup	0.14	0.14
	LR	2.5E-05	2.5E-05
RoBERTa	Epochs	8	12
	Batch	32	16
	Dropout	0.23	0.15
	Warmup	0.15	0.15
	LR	3E-05	7.8E-06

Table 2: Decoder-only model hyperparameters.

Parameter	Llama3	Mistral
Epochs	3	3
Batch size	16	8
Warmup ratio	0.1	0.05
Learning rate	1.00E-5	3.00E-4
Optimizer	adamw	paged_adamw*
Grad. accum.	2	4
LoRA r/α	64/16	64/16
LoRA dropout	0.1	0.1
Target modules	q,v,o	q,v

* 8-bit quantization; q,v,o: q_proj, v_proj, o_proj; weight decay: 0.001; max grad. norm: 1.0; scheduler: cosine w/ restarts.

Table 3: Prompt for context summarization.

Summarization Prompt
You are a helpful assistant. Provide a factual summarization around 150 words. Input claim: "{claim_text}" Relevant Context: {extracts_from_retrieved_wiki_pages} Generate a concise, objective summary to the provided claim based ONLY on the provided context.

3 Prompts

This section provides the complete prompt templates used in our experiments. Table 3 shows the prompt for generating context summaries from retrieved Wikipedia content, and Table 4 presents our prompt template. The baseline prompt uses the same structure but omits the “Additional information” component.

Table 4: Prompt template for verifiable claim detection. The baseline prompt excludes the highlighted components.

Prompt Template
Instruction: Determine if the input text contains verifiable claims. The input text contains verifiable claims if it makes specific factual statements that can be checked against evidence. Additional information may help clarify what the claim refers to, but base your decision primarily on whether the claim makes specific factual statements. If the input text contains claims that can be verified, respond "Yes". Otherwise, respond "No". Note: When in doubt, choose "Yes". In the end, respond only with 'Yes' for verifiable claims or 'No' for unverifiable claims. ### Input text: {claim_text} ### Additional information: {context_summary} ### Response: {Yes/No}

4 Context Summary Quality Assessment

To evaluate the quality of generated context summaries, we employed human annotators to assess 160 claim-summary pairs (40 samples $\times$ 2 datasets $\times$ 2 LLM-based generators) across three dimensions using a 3-point Likert scale (1=Poor, 2=Acceptable, 3=Good), as described in Section ?? . Since both datasets are publicly available, the evaluation posed minimal privacy or ethical concerns. Annotators were trained following the guidelines presented below to assess whether summaries provide useful context for determining claim verifiability, rather than assessing the truthfulness of the claims themselves.

Relevance. How relevant is the summary to the claim being assessed?

- Poor (1): Content is off-topic, unrelated to the claim, or discusses completely different or not mentioned entities/events.
- Acceptable (2): Provides some relevant background but includes unnecessary tangential information or misses some related aspects.
- Good (3): Content directly relates to the entities, events, or concepts mentioned in the claim; stays on topic.

Signal Clarity. Does the summary provide clear signals for determining verifiability, such as whether entities are real or fictional, events are documentable or hypothetical, statements are objective or subjective?

- Poor (1): Provides no clear signals about verifiability; leaves it unclear whether the claim can be fact-checked, e.g., only gives general background without addressing existence or documentability of specific entities or events.
- Acceptable (2): Provides some verifiability signals but they are implicit, incomplete, or ambiguous, e.g., confirms an entity exists but does not clarify whether the specific event is documentable.
- Good (3): Explicitly provides clear signals that help determine verifiability, e.g., confirms entities are real persons/organizations,

indicates whether events are recorded, and clarifies whether statements are objective or subjective.

Usefulness. How confident are you in determining claim verifiability after reading the summary?

- Poor (1): The summary does not help at all; you feel equally uncertain about the claim's verifiability as before reading it.
- Acceptable (2): The summary provides some help but leaves significant uncertainty; you would need additional information to make a confident verifiability judgment.
- Good (3): The summary significantly helps you determine whether the claim is verifiable or unverifiable; you feel confident in making this assessment based on the provided context.

References

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2623–2631.